CaneCCS-MLOps

An MLOps use case: CI/CD pipeline and monitoring for sugarcane CCS prediction

ISE-2 • ML DevOps Laboratory (UDSPC0732) • Academic Year 2026–27

Name	Niranjan	**PRN**	2324000035
Course	ML DevOps Laboratory	**Course Code**	UDSPC0732
Assessment	ISE-2 (10 marks)	**COs covered**	CO1 (L2), CO4 (L4)
Tools	Python, Flask, Git, GitHub, Jenkins, Docker, Prometheus, Grafana	**Date**	29 October 2026

1. Problem statement

Sugar mills pay farmers according to the **Commercial Cane Sugar (CCS %)** of each delivered batch, but the laboratory value is available only hours after the truck is unloaded. Cane also loses sugar every day it waits between harvest and crushing (field-to-mill delay), which is a recurring source of disputes in Maharashtra's sugar belt.

Goal. Build a small ML web service that estimates CCS % from fast weighbridge readings (brix, pol, fibre %, trash %, days since harvest, cane variety), and wrap it in a complete **CI/CD and monitoring** workflow so that every code change is tested, containerised, deployed and observed automatically.

Why this niche use case?

Most lab projects deploy a generic Iris or house-price model. Here the model is deliberately domain-specific (sugarcane agronomy), and the monitoring is domain-aware: it tracks *brix drift* (a change in incoming cane quality, e.g. after a bad monsoon) and *low-CCS batches*, not just HTTP errors. This is what separates ML monitoring from ordinary web monitoring.

Mapping to course outcomes

CO1 (L2)	Explain and configure the DevOps toolchain: Git, GitHub, Jenkins, Docker, and how they fit the ML lifecycle (Sections 4–8).
CO4 (L4)	Analyse a deployed ML service using monitoring tools: metrics design, dashboards, alert rules, drift signal (Sections 9–11).

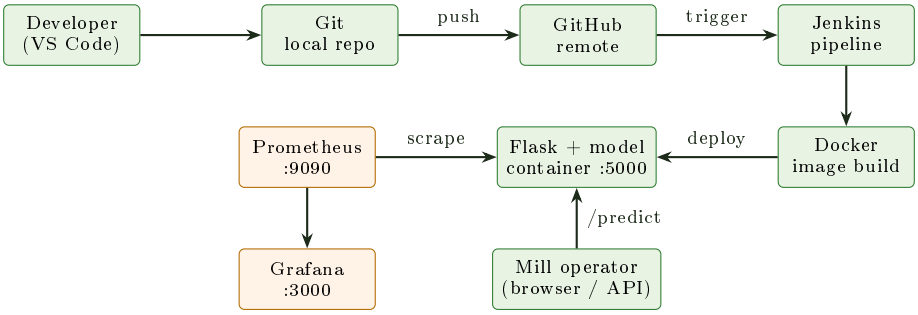
2. Model and data

Model: `GradientBoostingRegressor` (scikit-learn) predicting CCS %. **Inputs:** brix, pol, fibre %, trash %, days since harvest, variety (Co-86032, CoM-0265, Co-0238).

Data. A real mill dataset is proprietary, so `train.py` generates **synthetic** lab records (6,000 rows) from the standard Indian CCS relation $\mathrm{CCS}_{juice} = 0.73\,[\,\mathrm{pol} - 0.4(\mathrm{brix} - \mathrm{pol})\,]$, corrected for fibre and trash, a field-to-mill sugar-loss term (about 0.28 % per day, increasing with delay), a small variety effect and noise. The focus of this assignment is the *pipeline*, not model novelty, and this is disclosed so results are not mistaken for field data.

Metric (20% hold-out)	Value	Train rows
MAE	0.118 CCS points	4,800
R^2	0.9855	

3. System architecture



4. Tools: installation and verification

Tool	Role in this project	Verification command
Python 3.12	runtime for training and Flask	`python -version`
Flask 3.1	web/API layer	`python -c "import flask;print(flask.__version__)"`
Git	version control	`git -version`
GitHub	remote repo, webhook source	`git remote -v`
Docker Desktop	containerisation	`docker -version && docker run hello-world`
Jenkins (LTS)	CI/CD orchestration	open `http://localhost:8080`
Prometheus	metrics collection	open `http://localhost:9090/targets`
Grafana	dashboards	open `http://localhost:3000`

Screenshot to paste: terminal showing python, git and docker version outputs

Screenshot to paste: Jenkins login page / dashboard at localhost:8080 (after unlock + suggested plugins)

Jenkins setup checklist

1. Run Jenkins (Windows installer, or `docker run -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts`).
2. Unlock with the initial admin password, install *suggested plugins*, add *Pipeline*, *Git*, *GitHub* and *JUnit* if missing.
3. Make sure the Jenkins user can run `docker` and `python` (Windows: add both to the system PATH and restart the Jenkins service).

5. Flask application

Project layout

```
canesugar-mlops/
|-- app/main.py # Flask API + Prometheus metrics
|-- app/templates/index.html # prediction form
|-- train.py # trains and saves model + meta.json
|-- tests/test_app.py # 6 pytest unit tests
|-- Dockerfile docker-compose.yml Jenkinsfile
|-- prometheus/{prometheus.yml, alerts.yml}
`-- grafana/{provisioning/, dashboards/cane.json}
```

Endpoints

`GET /`	HTML form	operator UI
`POST /predict`	JSON → {ccs, grade}	validates input, returns 400 on bad data
`GET /health`	liveness probe	used by Docker and Jenkins smoke test
`GET /model-info`	model metadata	version, MAE, R^2
`GET /metrics`	Prometheus exposition	scraped every 5 s

app/main.py

```python
import json
import time
from collections import deque
from pathlib import Path

import joblib
import pandas as pd
from flask import Flask, Response, jsonify, render_template, request
from prometheus_client import (CONTENT_TYPE_LATEST, Counter, Gauge, Histogram,
                               generate_latest)

MODEL_DIR = Path(__file__).resolve().parent.parent / "model"

# ---- Prometheus metrics ------------------------------------------------
REQUESTS = Counter("cane_http_requests_total", "HTTP requests", ["endpoint", "status"])
LATENCY = Histogram("cane_request_latency_seconds", "Request latency", ["endpoint"],
                    buckets=(0.005, 0.01, 0.025, 0.05, 0.1, 0.25, 0.5, 1))
PRED_HIST = Histogram("cane_predicted_ccs", "Predicted CCS %", buckets=(6, 7, 8, 9, 10, 11, 12, 13))
PRED_TOTAL = Counter("cane_predictions_total", "Predictions served")
PRED_ERRORS = Counter("cane_prediction_errors_total", "Invalid or failed predictions")
BRIX_DRIFT = Gauge("cane_brix_drift_zscore", "Rolling mean brix vs training mean, in std units")
MODEL_INFO = Gauge("cane_model_info", "Model metadata", ["version", "algorithm"])
LOW_CCS = Counter("cane_low_ccs_batches_total", "Batches predicted below 9 CCS")


def create_app():
    app = Flask(__name__)
    model = joblib.load(MODEL_DIR / "ccs_model.joblib")
    meta = json.loads((MODEL_DIR / "meta.json").read_text())
    MODEL_INFO.labels(meta["model_version"], meta["algorithm"]).set(1)
    window = deque(maxlen=50)

    def timed(endpoint):
        def deco(fn):
            def wrapper(*a, **kw):
                t0 = time.perf_counter()
                resp = fn(*a, **kw)
                status = resp[1] if isinstance(resp, tuple) else 200
                LATENCY.labels(endpoint).observe(time.perf_counter() - t0)
                REQUESTS.labels(endpoint, str(status)).inc()
                return resp
            wrapper.__name__ = fn.__name__
            return wrapper
        return deco

    @app.get("/")
    @timed("index")
    def index():
        return render_template("index.html", varieties=meta["varieties"], meta=meta)

    @app.get("/health")
    @timed("health")
    def health():
        return jsonify(status="ok", model_version=meta["model_version"])

    @app.get("/model-info")
    @timed("model_info")
    def model_info():
        return jsonify(meta)

    @app.post("/predict")
    @timed("predict")
```

```
    def predict():
        try:
            d = request.get_json(force=True)
            variety = d["variety"]
            row = {
                "brix": float(d["brix"]), "pol": float(d["pol"]),
                "fibre_pct": float(d["fibre_pct"]), "trash_pct": float(d["trash_pct"]),
                "days_since_harvest": int(d["days_since_harvest"]),
                "variety_code": meta["varieties"].index(variety),
            }
            if not (10 <= row["brix"] <= 30) or row["pol"] > row["brix"]:
                raise ValueError("brix must be 10-30 and pol cannot exceed brix")
            ccs = float(model.predict(pd.DataFrame([row])[meta["features"]])[0])
        except (KeyError, ValueError, TypeError) as e:
            PRED_ERRORS.inc()
            return jsonify(error=str(e)), 400
        PRED_TOTAL.inc()
        PRED_HIST.observe(ccs)
        if ccs < 9:
            LOW_CCS.inc()
        window.append(row["brix"])
        BRIX_DRIFT.set(((sum(window) / len(window)) - meta["train_brix_mean"]) / meta["train_brix_std"])
        grade = "A" if ccs >= 10.5 else "B" if ccs >= 9.5 else "C"
        return jsonify(ccs=round(ccs, 3), grade=grade, model_version=meta["model_version"])

    @app.get("/metrics")
    def metrics():
        return Response(generate_latest(), mimetype=CONTENT_TYPE_LATEST)

    return app


app = create_app() if (MODEL_DIR / "ccs_model.joblib").exists() else None

if __name__ == "__main__":
    (app or create_app()).run(host="0.0.0.0", port=5000)
```

Screenshot to paste: browser at localhost:5000 showing the prediction form with a result

6. Git and GitHub

```
git init
git add .
git commit -m "Initial commit: CaneCCS MLOps pipeline"
git branch -M main
git remote add origin https://github.com/<your-username>/canesugar-mlops.git
git push -u origin main
```

Branch workflow used: feature branch → pull request → merge to `main`; the merge triggers Jenkins. A lightweight GitHub Actions workflow (`.github/workflows/ci.yml`) also runs training and tests on every push as a second, cloud-side check.

Screenshot to paste: GitHub repository page showing the project files and commit history

7. Docker

Listing 1: Dockerfile

```
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python train.py
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request;urllib.request.urlopen('http://localhost:5000/
    health')" || exit 1
CMD ["gunicorn", "-b", "0.0.0.0:5000", "-w", "1", "--threads", "4", "app.main:app"]
```

Listing 2: docker-compose.yml (app + Prometheus + Grafana)

```
services:
  cane-app:
    build: .
    image: canesugar-mlops:latest
    container_name: cane-app
    ports: ["5000:5000"]
    restart: unless-stopped

  prometheus:
    image: prom/prometheus:latest
    container_name: prometheus
    volumes:
      - ./prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
      - ./prometheus/alerts.yml:/etc/prometheus/alerts.yml:ro
    ports: ["9090:9090"]
    depends_on: [cane-app]

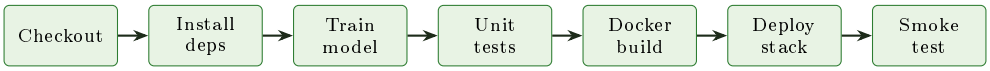
  grafana:
    image: grafana/grafana:latest
    container_name: grafana
    environment:
      - GF_SECURITY_ADMIN_USER=admin
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - ./grafana/provisioning:/etc/grafana/provisioning:ro
      - ./grafana/dashboards:/var/lib/grafana/dashboards:ro
    ports: ["3000:3000"]
    depends_on: [prometheus]
```

Design choice: one gunicorn worker with threads

`prometheus_client` keeps counters in process memory. With two gunicorn workers each worker reports its own numbers, so Prometheus sees values jump between workers. The container therefore uses `-w 1 -threads 4`, which keeps all metrics in one process (see Section 11).

Screenshot to paste: terminal: docker compose up -d –build, then docker ps showing 3 containers

8. Jenkins CI/CD pipeline



The pipeline is a *declarative* Jenkinsfile stored in the repository (pipeline-as-code). If tests fail the build stops, so a broken model never reaches the Docker stage.

Creating the job: New Item → *Pipeline* → Definition: *Pipeline script from SCM* → SCM: Git → repository URL → branch `*/main` → script path `Jenkinsfile`. Under *Build Triggers* enable *GitHub hook trigger* (or *Poll SCM* with `H/2 * * * *` when Jenkins is not publicly reachable).

Listing 3: Jenkinsfile

```
// Declarative CI/CD pipeline for CaneCCS MLOps
// Works on Linux agents (sh) and Windows agents (bat).
def run(cmd) { if (isUnix()) { sh cmd } else { bat cmd } }

pipeline {
    agent any
    options { timestamps(); timeout(time: 20, unit: 'MINUTES') }
    environment {
        IMAGE = "canesugar-mlops"
        TAG = "${env.BUILD_NUMBER}"
    }
    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('Install deps') {
            steps {
                run 'python -m venv venv'
                script {
                    if (isUnix()) { sh 'venv/bin/pip install -r requirements.txt' }
                    else { bat 'venv\\Scripts\\pip install -r requirements.txt' }
                }
            }
        }
        stage('Train model') {
            steps {
                script {
                    if (isUnix()) { sh 'venv/bin/python train.py' }
                    else { bat 'venv\\Scripts\\python train.py' }
                }
            }
        }
        stage('Unit tests') {
            steps {
                script {
                    if (isUnix()) { sh 'venv/bin/python -m pytest -q --junitxml=report.xml' }
                    else { bat 'venv\\Scripts\\python -m pytest -q --junitxml=report.xml' }
                }
            }
            post { always { junit 'report.xml' } }
        }
        stage('Docker build') {
            steps { run "docker build -t ${IMAGE}:${TAG} -t ${IMAGE}:latest ." }
        }
        stage('Deploy stack') {
            steps { run 'docker compose up -d --build' }
        }
        stage('Smoke test') {
            steps {
                script {
                    sleep 8
                    if (isUnix()) { sh 'curl -f http://localhost:5000/health' }
                    else { bat 'curl -f http://localhost:5000/health' }
                }
            }
        }
    }
    post {
        success { echo "Build ${env.BUILD_NUMBER}: CaneCCS deployed. Grafana :3000, Prometheus :9090" }
        failure { echo "Build ${env.BUILD_NUMBER} failed - check stage logs" }
    }
}
```

Screenshot to paste: Jenkins job configuration page (Pipeline script from SCM)

Screenshot to paste: Jenkins Stage View with all stages green

Screenshot to paste: Jenkins Console Output ending with "Finished: SUCCESS"

9. Monitoring: Prometheus and Grafana

Metrics exposed by the application

Metric	Type	Why it matters
`cane_http_requests_total{endpoint,status}`	Counter	traffic and error ratio
`cane_request_latency_seconds`	Histogram	p95 latency SLO
`cane_predictions_total`	Counter	throughput
`cane_prediction_errors_total`	Counter	bad client input
`cane_predicted_ccs`	Histogram	prediction distribution (output drift)
`cane_brix_drift_zscore`	Gauge	rolling mean brix of last 50 requests vs. training mean, in std units (input drift)
`cane_low_ccs_batches_total`	Counter	business signal: batches predicted below 9 CCS
`cane_model_info`	Gauge	which model version is live

Listing 4: prometheus/prometheus.yml

```
global:
  scrape_interval: 5s
  evaluation_interval: 5s
rule_files:
  - /etc/prometheus/alerts.yml
scrape_configs:
  - job_name: cane-app
    metrics_path: /metrics
    static_configs:
      - targets: ["cane-app:5000"]
```

Listing 5: prometheus/alerts.yml

```
groups:
  - name: cane-ml-alerts
    rules:
      - alert: CaneAppDown
        expr: up{job="cane-app"} == 0
        for: 15s
        labels: {severity: critical}
        annotations: {summary: "CaneCCS service is down"}
      - alert: BrixInputDrift
        expr: abs(cane_brix_drift_zscore) > 1.5
        for: 30s
        labels: {severity: warning}
        annotations: {summary: "Incoming brix distribution has drifted from training data"}
```

```
    - alert: HighErrorRate
      expr: rate(cane_prediction_errors_total[1m]) > 0.2
      for: 30s
      labels: {severity: warning}
      annotations: {summary: "Many invalid prediction requests"}
```

Grafana

Grafana is *provisioned as code*: the data source (`grafana/provisioning/datasources/ds.yml`) and a six-panel dashboard (`grafana/dashboards/cane.json`) load automatically at start-up, so the monitoring view is reproducible and not hand-built. Login `admin/admin` at `localhost:3000`, dashboard *CaneCCS ML Monitoring*.

Panel	PromQL
Predictions per minute	`rate(cane_predictions_total[1m])*60`
p95 predict latency	`histogram_quantile(0.95, sum(rate(cane_request_latency_seconds_bucket {endpoint="predict"}[1m])) by (le))`
Brix drift (z-score)	`cane_brix_drift_zscore`
Invalid requests / min	`rate(cane_prediction_errors_total[1m])*60`

Screenshot to paste: Prometheus Status → Targets page showing cane-app as UP

Screenshot to paste: Grafana dashboard "CaneCCS ML Monitoring" with live panels

10. Testing and results

Unit tests (pytest)

```
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.7.0 -- /usr/bin/python3
collecting ... collected 6 items
tests/test_app.py::test_health PASSED [ 16%]
tests/test_app.py::test_predict_ok PASSED [ 33%]
tests/test_app.py::test_staleness_lowers_ccs PASSED [ 50%]
tests/test_app.py::test_rejects_pol_above_brix PASSED [ 66%]
tests/test_app.py::test_rejects_missing_field PASSED [ 83%]
tests/test_app.py::test_metrics_exposed PASSED [100%]
============================== 6 passed in 1.94s ===============================
```

API calls against the running service

```
$ curl -s localhost:5000/health
{"model_version":"2026.10.27","status":"ok"}

$ curl -X POST /predict # Co-86032, fresh, brix 19.5
{"ccs":9.606,"grade":"B","model_version":"2026.10.27"}
$ curl -X POST /predict # Co-0238, 5 days after harvest
{"ccs":8.582,"grade":"C","model_version":"2026.10.27"}
$ curl -X POST /predict # invalid: pol > brix
{"error":"brix must be 10-30 and pol cannot exceed brix"}
```

Observations: the same cane (brix 19.5, pol 17.2) predicts 9.61 CCS when one day old (grade B) but a different variety held five days drops to 8.58 (grade C), reproducing the field-to-mill loss the model was built to capture. Invalid input (pol greater than brix) is rejected with HTTP 400.

Drift experiment

After normal traffic I sent 30 high-brix batches (brix 23.5, against a training mean of 19.5). The service then exposed:

```
cane_http_requests_total{endpoint="health",status="200"} 1.0
cane_http_requests_total{endpoint="predict",status="200"} 32.0
cane_http_requests_total{endpoint="predict",status="400"} 1.0
cane_predictions_total 32.0
cane_prediction_errors_total 1.0
cane_brix_drift_zscore 2.3352095059412146
cane_model_info{algorithm="GradientBoostingRegressor",version="2026.10.27"} 1.0
```

The brix drift z-score of about 2.3 exceeds the 1.5 threshold in `alerts.yml`, so the `BrixInputDrift` alert would fire after 30 s, signalling that the input distribution has moved away from what the model was trained on and retraining should be considered.

11. Problems faced and how they were solved

1. **Metrics did not add up.** 32 valid predictions were sent but `/metrics` showed 18. Cause: gunicorn ran two worker processes, each with its own in-memory counters. Fix: one worker with four threads (`-w 1 -threads 4`); afterwards the counters matched exactly (32 predictions, 1 error).
2. **Windows vs. Linux Jenkins agents.** `sh` fails on Windows agents. The Jenkinsfile uses `isUnix()` to pick `sh` or `bat`.
3. **Model missing in a fresh clone.** The `.joblib` file is git-ignored, so the pipeline (and Dockerfile) retrain it with `train.py`, keeping the repository small and builds reproducible.
4. **Grade thresholds.** Initial A/B/C cut-offs never produced grade A on realistic inputs; they were recalibrated against the model's output range.

12. Conclusion

The project delivers a niche, domain-specific ML service together with a full DevOps loop: code in Git/GitHub, an automated Jenkins pipeline that trains, tests, builds and deploys a Docker image, and a Prometheus/Grafana stack that watches both system health and ML-specific signals (input drift, prediction distribution, low-quality batches). Future work: multiprocess Prometheus mode for horizontal scaling, Alertmanager e-mail/Slack notifications, model registry (MLflow) and a scheduled retraining job triggered by the drift alert.

References

- Flask docs: https://flask.palletsprojects.com Jenkins docs: https://www.jenkins.io/doc/book/pipeline/
- Docker docs: https://docs.docker.com Prometheus: https://prometheus.io/docs Grafana: https://grafana.com/docs
- prometheus_client (Python): https://github.com/prometheus/client_python